

```

1  from multiprocessing import Process, Manager
2  import os
3  import numpy as np
4  import time
5  import random
6  from PIL import Image
7  import pandas
8
9  # Il y a beaucoup de commentaires mais le code tient en 10 lignes
10 def div_P(poly):
11     "Calcule le reste de la div de poly (en binaire) par P"
12     if poly < 2**8:
13         return poly
14     # On effectue d'abord la div par X^8
15     quotient = poly >> 8 # Ça équivaut à se donner le quotient par X^8
16     reste = poly - (quotient * 2**8)
17     # Pour chaque puissance 8+n : X^8 X^n = (P - X^8) X^n
18     # Ce qui revient à augmenter les puissances de P-X^8 de n puis à l'ajouter au reste du polynôme.
19     # Puisque en python, + = - = ^, on "shift" P ^ 2**8 (py, ^ vu comme -) par n puis on ajoute (^) ça au reste
20     n = 0
21     # P0 = X^8 + X^7 + X^2 + X + 1 = 0b110000111
22     # Comme le seul endroit où on l'utilise, on l'utilise - X^8, on pose direct :
23     P0 = 0b110000111 ^ 2**8
24     for coeff in bin(quotient)[1:-1]:
25         # Si coeff = 1, tout ce passe bien, si = 0, ^0 ne fait rien
26         reste ^= (P0 << n) * int(coeff)
27         n += 1
28     # Comme rien n'empêche que les n soient très grands et donc que le reste ait toujours un degré ≥ 8.
29     return div_P(reste)
30
31
32 #####
33 # F256 #
34 #####
35
36 class F256:
37     # Attention ! F256 prend en entrée un polynôme et non pas une puissance de X
38     def __init__(self, polynome):
39         self.poly = div_P(polynome)
40     def __add__(self, other):
41         return F256(self.poly ^ other.poly)
42     # Nous sommes en caractéristique 2 donc
43     def __sub__(self, other):
44         return F256(self.poly ^ other.poly)
45     def __neg__(self):
46         return self
47     def __int__(self):
48         return self.poly
49     def __str__(self):
50         # Finalement, on affiche juste le polynôme et non pas la puissance de X
51         return str(self.poly)
52     def __repr__(self):
53         return str(self.poly)
54     def __eq__(self, other):
55         return self.poly == other.poly
56
57
58 # D'un POV purement Python, on a besoin d'avoir défini F256 avant de définir create_table
59 # Mais pour définir la méthode __mul__ de F256, on a besoin de create_table.
60 def create_tables():
61     table_log = {}
62     table_exp = {}
63     for puissance in range(2**8 - 1):
64         # 2 représente X ici
65         table_log[F256(2**puissance).poly] = puissance
66         table_exp[puissance] = F256(2**puissance)
67     return table_log, table_exp
68
69 # La table des puissances
70 table_log, table_exp = create_tables()
71
72 def mulF256(self, other):
73     try:

```

```

74         # nouvelle multiplication -> on additionne les puissances de X
75         # Il faut faire ce test avant car 0 n'est pas une puissance de X
76         if 0 in [self.poly, other.poly]:
77             return F256(0)
78         return table_exp[(table_log[self.poly] + table_log[other.poly]) % 255]
79     except:
80         raise Exception("Il faut multiplier par un autre nombre de F256 !")
81
82 def powF256(self, scalaire):
83     if scalaire == 0:
84         return F256(1)
85     elif self.poly == 0:
86         return self
87     else:
88         return table_exp[(table_log[self.poly] * scalaire) % 255]
89
90 def truedivF256(self, other):
91     if other.poly == 0:
92         raise ZeroDivisionError()
93     elif self.poly == 0:
94         return self
95     else:
96         return table_exp[(table_log[self.poly] - table_log[other.poly]) % 255]
97
98 setattr(F256, "__mul__", mulF256)
99 setattr(F256, "__pow__", powF256)
100 setattr(F256, "__truediv__", truedivF256)
101
102
103 #####
104 # FONCTIONS GÉNÉRALES SUR LES POLYNÔMES #
105 #####
106
107 def clean(poly):
108     copie = poly.copy()
109     if copie != []:
110         while copie[-1] == F256(0):
111             copie.pop()
112             if copie == []:
113                 break
114     return copie
115
116 def prod(P, Q):
117     n1, n2 = len(P), len(Q)
118     PQ = [F256(0)] * (n1 + n2 - 1)
119     for k in range(n1):
120         Pk = P[k]
121         for i in range(n2):
122             PQ[k + i] += Pk * Q[i]
123     return clean(PQ)
124
125
126 # On est en caractéristique 2 donc pour soustraire des polynômes, on les additionne
127 def somme(P, Q):
128     n0 = max(len(P), len(Q))
129     P += [F256(0)] * (n0 - len(P))
130     Q += [F256(0)] * (n0 - len(Q))
131     resultat = [P[i] + Q[i] for i in range(n0)]
132     return clean(resultat)
133
134 def generateur(t):
135     G = [F256(1)]
136     for i in range(2 * t):
137         G = prod(G, [table_exp[i], F256(1)])
138     return G
139
140 def div_euclid(P, Q):
141     P = clean(P)
142     Q = clean(Q)
143     degP, degQ = len(P) - 1, len(Q) - 1
144     if degP < degQ:
145         return [[], P]
146     if degQ == -1:
147         raise ZeroDivisionError()

```

```

148 quotient = []
149 c_dom_Q = Q[-1]
150 for i in range(degP, degQ - 1, -1):
151     # Il est possible lors de la division euclidienne qu'en retirant sub à P,
152     # on élimine aussi le terme d'après donc on skip le terme nul si ça arrive
153     if i < len(P):
154         mult = P[i] / c_dom_Q
155         sub = [F256(0)] * (i - degQ) + [mult * c_Q for c_Q in Q]
156         P = somme(P, sub)
157         quotient.insert(0, mult)
158     return [quotient, P]
159
160
161 #####
162 # ENCODAGE INDIVIDUEL #
163 #####
164
165 def encodage_v2(message, k, t, results, i):
166     P = [F256(0) for i in range(2 * t + 1)]
167     P[-1] = F256(1)
168     controle = div_euclid(prod(message, P), generateur(t))[1]
169     transmis = somme(prod(message, P), controle)
170     transmis += [F256(0)] * (k + 2*t - len(transmis)) # On fait en sorte qu'il est exactement k+2t coefficients
171     results[i] = transmis
172
173 def evaluation(P, a):
174     s = F256(0)
175     for i in range(len(P)):
176         s += P[i] * (a**i)
177     return s
178
179
180 #####
181 # DECODAGE INDIVIDUEL #
182 #####
183
184 def identite(n):
185     mat = np.full((n, n), F256(0), dtype=F256)
186     for i in range(n):
187         mat[i, i] = F256(1)
188     return mat
189
190 def algo_du_pivot(mat):
191     cop = mat.copy()
192     n = cop.shape[0]
193     n_pivots_prec = 0
194     inv = identite(n)
195     if n != cop.shape[1]:
196         raise Exception("Elle est pas carrée !")
197     for col in range(n):
198         pivot = None
199         # À chaque fois qu'on fait un pivot, on regarde plus que les lignes au dessus
200         for ligne in range(n_pivots_prec, n):
201             pivot = cop[ligne, col]
202             if pivot != F256(0):
203                 # On passe à la suite
204                 break
205         if pivot == F256(0):
206             # Dans ce cas, la matrice n'est pas inversible
207             return False
208         # On échange les lignes "ligne" et "n_pivots_prec"
209         # Il faut mettre des doubles crochets bien échanger les lignes
210         cop[[ligne], cop[[n_pivots_prec]]] = cop[[n_pivots_prec], cop[[ligne]] #type:ignore
211         inv[[ligne], inv[[n_pivots_prec]]] = inv[[n_pivots_prec], inv[[ligne]] #type:ignore
212         # Opération Li <- Li - coeff/pivot Lj
213         # et Lj <- coeff/pivot Lj
214         for i in range(n):
215             if cop[i, col] != F256(0):
216                 if i == n_pivots_prec:
217                     cop[i] /= pivot
218                     inv[i] /= pivot
219                     pivot = F256(1)
220             else:
221                 coeff = cop[i, col] / pivot

```

```

222         cop[i] -= np.multiply(coeff, cop[n_pivots_prec])
223         inv[i] -= np.multiply(coeff, inv[n_pivots_prec])
224         n_pivots_prec += 1
225     return inv
226
227 def syndromes(recu, t):
228     return [evaluation(recu, table_exp[i]) for i in range(2 * t)]
229
230 def det_poly_correcteur(t, synds):
231     for nu in range(t, 0, -1):
232         mat = np.full((nu, nu), F256(0), dtype=F256)
233         for ligne in range(nu):
234             for col in range(nu):
235                 mat[ligne, col] = synds[nu + ligne - col - 1]
236         inv = algo_du_pivot(mat)
237         if inv is not False: # inv = False si la matrice n'est pas inversible
238             mat_col = np.array(syns[nu:2 * nu]).reshape((nu, 1))
239             lambdas = (inv @ mat_col).reshape((1, nu)).tolist()
240             return [F256(1)] + lambdas[0]
241         raise Exception("Ya jamais de solution, c'est pas normal")
242
243 def det_ir_xr(poly_lambda):
244     ir, xr = [], []
245     for puissance in range(255):
246         if evaluation(poly_lambda, table_exp[puissance]) == F256(0):
247             ir.append((-puissance) % 255)
248             xr.append(table_exp[(-puissance) % 255])
249     return ir, xr
250
251 def det_E(ir, xr, synds, k, t):
252     nu = len(xr)
253     mat_xr = np.full((nu, nu), F256(0), dtype=F256)
254     for ligne in range(nu):
255         for col in range(nu):
256             mat_xr[ligne, col] = xr[col] ** ligne
257     mat_col = np.array(syns[:nu]).reshape((nu, 1))
258     inv_xr = algo_du_pivot(mat_xr)
259     yr = (inv_xr @ mat_col).reshape((1, nu)).tolist()[0]
260     E = []
261     for i in range(k + 2 * t):
262         if i in ir:
263             E.append(yr.pop())
264         else:
265             E.append(F256(0))
266     return E
267
268 def decodage_v2(recu, k, t, results, i):
269     synds = syndromes(recu, t)
270     correct = True
271     for s in synds:
272         if s != F256(0):
273             correct = False
274             break
275     if correct:
276         results[i] = recu[-k:]
277     else:
278         poly_correcteur = det_poly_correcteur(t, synds)
279         ir, xr = det_ir_xr(poly_correcteur)
280         E = det_E(ir, xr, synds, k, t)
281         s = somme(recu, E)[-k:]
282         s += [F256(0)] * (k - len(s))
283         results[i] = s
284
285
286 #####
287 # MULTIPROCESS #
288 #####
289
290 # Execute sur chaque bloc une fonction f qui respecte le bon format
291 # Utile pour l'encodage et le decodage
292 def multiprocess(blocs, f, t, n_process, silent):
293     n_blocs = len(blocs)
294     blocs_traites = [None] * n_blocs
295     # Gestion des processus en parallèle

```

```

296 process = [None] * n_process
297 manager = Manager()
298 results = manager.list([None] * n_process)
299 coords_list = [None] * n_process
300
301 for g in range(n_blocs // n_process):
302     if g == 0:
303         debut = time.time()
304     # Lancement de chaque processus
305     for i in range(n_process):
306         empla = n_process * g + i
307         coords_list[i] = blocs[empla][1]
308         k_i = coords_list[i][2] * coords_list[i][3]
309         process[i] = Process(target=f, args=(blocs[empla][0].tolist(), k_i, t, results, i))
310         process[i].start()
311     for i in range(n_process):
312         process[i].join()
313         empla = n_process * g + i
314         # Stockage des résultats
315         blocs_traites[empla] = (np.array(results[i]), coords_list[i]) #type:ignore
316     if g == 0:
317         fin = time.time()
318         if not silent:
319             print("Temps estimé :", int((fin - debut) * (n_blocs // n_process)), "secondes.") #type:ignore
320
321 # S'il reste quelques processus on les fait (n_process peut ne pas diviser n_blocs)
322 m = n_blocs % n_process
323 for g in range(m):
324     for i in range(m):
325         empla = n_blocs - i - 1
326         coords_list[i] = blocs[empla][1]
327         k_i = coords_list[i][2] * coords_list[i][3]
328         process[i] = Process(target=f, args=(blocs[empla][0].tolist(), k_i, t, results, i))
329         process[i].start()
330     for i in range(m):
331         process[i].join()
332         empla = n_blocs - i - 1
333         blocs_traites[empla] = (np.array(results[i]), coords_list[i]) #type:ignore
334
335     return blocs_traites
336
337
338 #####
339 # ENCODAGE IMAGE #
340 #####
341
342 # Récupère une image d'une URL et nous renvoie une matrice d'éléments de F256 la représentant.
343 def extract_img(url):
344     # img = Image.open(urlopen(url))
345     img = Image.open(url)
346     largeur, hauteur = img.size
347     pixels = list(img.get_flattened_data())
348     img.close()
349     # RGB fait tripler la largeur de l'image
350     Fpixels = np.full((hauteur, largeur * 3), F256(0), dtype=F256)
351     for i in range(len(pixels)):
352         x = i % largeur
353         y = i // largeur
354         for c in range(3):
355             Fpixels[y, 3 * x + c] = F256(pixels[i][c]) #type: ignore
356     return Fpixels
357
358 # NOTE:
359 # Ce n'est pas un problème que la taille des blocs soit différente
360 # alors qu'ils ont le même t.
361
362 # Prend une matrice de F256 et la divise en blocs.
363 # Les blocs peuvent ne pas quadriller parfaitement.
364 # Les blocs sont des blocs de composantes de couleur et non de pixels
365 def diviser_blocs(img_src, blocs_x, blocs_y):
366     img_y, img_x = img_src.shape
367     x, y = 0, 0
368     blocs = []
369     while True:

```

```

370 # Les coordonnées et dimensions réelles du bloc (tenant compte des bords)
371 coords = (
372     x, y,
373     blocs_x if x + blocs_x <= img_x else img_x - x,
374     blocs_y if y + blocs_y <= img_y else img_y - y
375 )
376 bloc_xy = img_src[y:y+coords[3], x:x+coords[2]]
377 blocs.append((bloc_xy.flatten(), coords))
378 # Coin en bas à droite
379 if x + blocs_x >= img_x and y + blocs_y >= img_y:
380     break
381 # Côté de droite
382 if x + blocs_x < img_x:
383     x += blocs_x
384 else:
385     x, y = 0, y + blocs_y
386 return blocs
387
388 def encoder_blocs(blocs, t, n_process, silent):
389     return multiprocessing(blocs, encodage_v2, t, n_process, silent)
390
391 #####
392 # DÉCODAGE IMAGE #
393 #####
394
395 # Changer un certain % de valeurs de couleurs aléatoirement
396 def bruit(blocs_encodes, pourcent):
397     # On calcule dans un premier temps la taille réelle du message transmis
398     img_trans_size = 0
399     chg = 0
400     array_bruite = []
401     for bloc, coords in blocs_encodes:
402         img_trans_size += len(bloc)
403         array_bruite.append((bloc.copy(), coords))
404     # On s'assure de modifier le bon pourcentage en:
405     # ne modifiant qu'une seule valeur de rgb
406     # en vérifiant qu'on n'a pas déjà modifié le pixel
407     while chg < img_trans_size * pourcent:
408         b_rand = random.randint(0, len(blocs_encodes)-1)
409         val_rand = random.randint(0, len(blocs_encodes[b_rand][0])-1)
410         if array_bruite[b_rand][0][val_rand] == blocs_encodes[b_rand][0][val_rand]:
411             array_bruite[b_rand][0][val_rand] = F256(random.randint(0, 255))
412             chg += 1
413     return array_bruite
414
415 def decoder_blocs(blocs, t, n_process, silent):
416     return multiprocessing(blocs, decodage_v2, t, n_process, silent)
417
418 # On récupère une image en bloc et on la reconstruit selon les bonnes dimensions toujours en F256
419 def recreer_img(blocs, img_x, img_y):
420     img = np.full((img_y, img_x), 0, dtype=F256)
421     for bloc, coords in blocs:
422         img[coords[1]:coords[1] + coords[3], coords[0]:coords[0] + coords[2]] = bloc.reshape((coords[3], coords[2]))
423     return img
424
425 # On récupère une image en F256 et on fait une image PIL
426 def PIL_img(img):
427     img_y, img_x = img.shape
428     if img_x % 3 != 0:
429         raise Exception("R, G et B ne sont pas à la suite")
430     rgb_img = np.full((img_y, img_x // 3, 3), 0)
431     for y in range(img_y):
432         for x in range(img_x // 3):
433             for c in range(3):
434                 rgb_img[y, x, c] = img[y, 3 * x + c].poly
435     PIL_img = Image.fromarray(rgb_img.astype("uint8"), "RGB")
436     return PIL_img
437
438 # On récupère une image résultat en F256 et l'image de base (en F256)
439 # et on renvoie le % d'erreur en couleur et en pixel
440 def erreur(final, initial):
441     compte_val = 0
442     compte_pixel = 0

```

```

444     if (len(final), len(final[0])) != (len(initial), len(initial[0])):
445         raise Exception("Les tailles ne correspondent pas ! (force à toi)")
446     for y in range(len(final)):
447         pixel = 0
448         erreur_pixel = False
449         for x in range(len(final[0])):
450             if final[y][x] != initial[y][x]:
451                 compte_val += 1
452                 if not erreur_pixel:
453                     erreur_pixel = True
454                     compte_pixel += 1
455             pixel += 1
456             if pixel == 3:
457                 pixel = 0
458                 erreur_pixel = False
459     size = len(final) * len(final[0])
460     return (compte_val / size, compte_pixel / size)
461
462 #####
463 # ANCIENNE VERSION #
464 #####
465
466 def encodage_v1(u):
467     k = u.shape[0]
468     a = np.zeros(255, dtype=F256)
469     for i in range(255):
470         c = F256(0)
471         for j in range(k):
472             c += u[j] * table_exp[(i * j) % 255]
473         a[i] = c
474     return a
475
476 def combi_random(jusqua, k, iter_max):
477     for i in range(iter_max):
478         yield random.sample(range(jusqua), k)
479
480 # Pour la suite, il est nécessaire que decodage ne renvoie pas de données
481 # mais modifie une liste en argument à l'emplacement prévu
482 def decodage_v1(w, k, lim_pointe, lim_max, liste, emplacement):
483     udict = {}
484     # Pour chaque façon de prendre k équations parmi les 255
485     for kcombi in combi_random(255, k, lim_max):
486         w_k = np.array([w[x] for x in kcombi])
487         matrice_systeme = np.zeros((k, k), dtype=F256)
488         for i in range(len(kcombi)):
489             for puissance in range(k):
490                 matrice_systeme[i, puissance] = table_exp[(kcombi[i] * puissance) % 255]
491         u = algo_du_pivot(matrice_systeme) @ w_k
492         # On convertit u en un object "hashable"
493         hash_u = tuple([int(i) for i in u])
494         if hash_u not in udict:
495             udict[hash_u] = 0
496             udict[hash_u] += 1
497         # Dès que l'on a plus de "limite" équations menant au même vecteur on s'arrête
498         max_compte = 0
499         max_u = None
500         for u in udict:
501             if udict[u] > max_compte:
502                 max_compte = udict[u]
503                 max_u = u
504         if max_compte >= lim_pointe:
505             liste[emplacement] = np.array(max_u)
506             return
507     raise Exception("pas trouvé")

```